```
    flit_server.check_for_mentions(self)
    flit_server.follower_list(user)
    user.update_attributes(:flit_count => user.flit_count + 1)
    # and so it goes …
  end
end
```

Within this tangled mess, you must add a new feature: if a flit contains text of the form $username, the user in question must be informed of the message. You could just add another if statement in the long line of if statements already in the method, but then it would be hard to test the new behavior without testing all the process_flit code, which brings in a lot of other stuff. (In real life this method could be 300 lines, and for all you know it could invoke PayPal.)

Instead, add a method check_for_dollar_sign, and then call that method in the appropriate place inside process_flit, writing the new method using regular TDD. If you're feeling adventurous and it seems plausible, a mock test to confirm that process_flit calls check_for_dollar_sign might also be appropriate.

If you're adding or extracting a lot of functionality, you might consider creating your own separate class rather than just a method. One sign that a new class is warranted is if it passes the same set of instance variables to multiple methods. I'm a big fan of classes that represent processes and replace long complex methods. For testing purposes, moving new code to a new class can make testing the new code easier because the new code is less dependent on the existing application.

Although this technique helps you make a clean break from the legacy code, it has the short-term effect of making the code more opaque. In the words of Michael Feathers, "When you break dependencies in legacy code, you often have to suspend your sense of aesthetics a bit." You know how when you're cleaning off your desk, you have an intermediate stage in which the room is covered in piles of paper and it looks like an even bigger mess? Or is that just me? In any case, you're in an intermediate state here, between the undifferentiated mass of the original and the nicely factored and organized new version. Building up the test suite one broken dependency at a time moves you steadily toward cleaner code.

Legacy Databases, Testing, and You

If your legacy application has only a nodding relationship with Rails common standards, chances are the database is also a mess. Many issues that plague a legacy database can be frustrating (such as odd naming conventions or